

Segmentación Semántica y Control Autónomo para un Vehículo Robot con Jetson Nano

Juan José Prado Neira Javier Rego Bargos
Trabajo de Visión Avanzada, TFM

Resumen—Este artículo presenta una práctica completa de visión artificial aplicada a la conducción autónoma de un vehículo robot basado en Jetson Nano. El sistema segmenta en tiempo real una zona transitable y obstáculos mediante una U-Net ligera entrenada con un conjunto de datos propio. La salida de la red se integra en una arquitectura distribuida donde la Jetson captura y aplica comandos, mientras que el portátil ejecuta la inferencia, calcula el desplazamiento lateral y genera las acciones de dirección y velocidad. El modelo final alcanza un Dice de validación de 0.9519 y queda exportado a ONNX para su integración con OpenCV DNN u ONNX Runtime. La contribución principal es una cadena funcional que cubre anotación, entrenamiento, despliegue y control reactivo ante obstáculos.

Palabras clave—visión artificial, segmentación semántica, U-Net, Jetson Nano, ONNX, robótica móvil.

I. INTRODUCCIÓN

La conducción autónoma en plataformas robóticas de bajo coste exige combinar percepción visual, comunicación de baja latencia y control robusto. En este contexto, la segmentación semántica resulta especialmente adecuada porque proporciona una descripción densa de la escena: cada píxel puede clasificarse como parte del circuito, obstáculo o fondo. Esta información permite extraer una referencia geométrica sencilla, como el centro de la zona transitable, sin depender de detectores de bordes frágiles ni de reglas específicas para una iluminación concreta.

La práctica desarrollada implementa un prototipo de extremo a extremo para un coche robot con Jetson Nano. El objetivo no es entrenar un modelo genérico de conducción, sino construir un sistema reproducible para seguir un circuito interior y reaccionar ante cajas situadas en la trayectoria. Para ello se crea un conjunto de datos propio, se entrena una U-Net compacta de dos salidas y se integra el modelo en un flujo distribuido de vídeo y control. Esta separación permite ejecutar en el portátil el cómputo pesado de inferencia, manteniendo en la Jetson la captura de cámara y la aplicación final de los comandos al vehículo.

II. METODOLOGÍA

El flujo de trabajo se dividió en cuatro etapas: adquisición de datos, entrenamiento, exportación del modelo e integración con el sistema de control. Primero se capturaron imágenes reales del circuito desde una perspectiva baja, similar a la del vehículo. Las muestras se anotaron con MakeSense en dos clases: *circuito*, que representa la zona transitable, y *caja*, que representa obstáculos. A partir del fichero COCO

exportado se generaron máscaras binarias independientes para cada clase.

El flujo de entrenamiento toma como referencia la práctica DeepLabv3+ del Tema 5 [1], centrada en dataloaders, Dice+BCE e IoU para segmentación semántica, y los fundamentos DeepLabv3/v3+ [2], [3]. Para este prototipo se eligió una U-Net ligera, inspirada en la arquitectura encoder-decoder con conexiones de salto propuesta por Ronneberger *et al.* [4]. La red recibe una imagen RGB de 256×256 píxeles y produce dos mapas de logits: circuito y obstáculos. El entrenamiento se realizó en PyTorch con Adam y una pérdida BCEWithLogits + Dice Loss ponderada para la clase de obstáculo.

La salida entrenada se exportó a ONNX para separar el entrenamiento del entorno de ejecución. En el runtime de visión, el modelo puede cargarse con OpenCV DNN y, si ese backend falla, con ONNX Runtime. Tras la inferencia se aplica una sigmoide por canal, umbrales configurables y operaciones morfológicas sobre la máscara del circuito. El centro de la zona transitable se calcula en una región de interés inferior de la imagen, donde la perspectiva del vehículo proporciona la información más relevante para la dirección inmediata.

III. ARQUITECTURA DEL SISTEMA

El sistema completo está organizado en cuatro bloques de software. El módulo *vision* procesa cada frame, ejecuta la segmentación y construye la información de carril. El módulo *conexion* implementa el transporte de vídeo, el receptor local y la orquestación del arranque remoto. El módulo *control* define el protocolo de comandos y el cliente de control manual. Finalmente, *jetson* contiene los procesos remotos que capturan la cámara y aplican las órdenes al vehículo.

Durante la ejecución, la Jetson envía frames JPEG al portátil mediante UDP. La configuración del repositorio fija una resolución de captura de 512×288 y una frecuencia objetivo de 12 FPS. El receptor decodifica cada imagen, obtiene las máscaras, suaviza el desplazamiento lateral con una media exponencial y calcula una acción automática normalizada. La orden se devuelve a la Jetson como un mensaje JSON por línea con los campos *steering*, *throttle*, modo y razón de la decisión.

La política de autonomía combina seguimiento de circuito y evitación de obstáculos. Si existe centro de carril, el error lateral se normaliza respecto a la mitad del ancho de la imagen y se transforma en dirección mediante una ganancia proporcional y un término de curvatura. Cuando la máscara

Tabla I
RESUMEN TÉCNICO DEL SISTEMA DESARROLLADO.

Elemento	Valor
Dataset	57 imágenes, 100 anotaciones
Clases	círculo, caja
Resolución original	640 × 360 píxeles
Partición	45 entrenamiento, 12 validación
Modelo	U-Net ligera, 7.763.074 parámetros
Entrada / salida	3 × 256 × 256 / 2 × 256 × 256
Optimizador	Adam, <i>batch size</i> 8
Pérdida	BCEWithLogits + Dice Loss
Mejor checkpoint	Época 78, Dice 0.9519
Despliegue	ONNX con OpenCV DNN/ONNX Runtime
Control	<i>steering, throttle</i> , parada/evasión



Figura 1. Ejemplo de inferencia: zona transitable segmentada en verde y obstáculo en rojo.

de obstáculo supera umbrales de área y tamaño dentro de la región frontal, el sistema decide entre reducir velocidad, rodear el obstáculo o detenerse. Si se pierde el centro del círculo, la política configurada detiene el vehículo para evitar acciones ciegas.

IV. RESULTADOS

El entrenamiento produjo un checkpoint seleccionado por Dice de validación. El mejor modelo se guardó en la época 78 con un Dice de 0.9519, lo que indica alta coincidencia entre las máscaras predichas y las anotaciones de validación. La métrica debe interpretarse dentro del alcance del conjunto de datos: se trata de un escenario controlado, con 57 imágenes y una división de validación de 12 muestras. Por tanto, el resultado confirma la viabilidad de la práctica, pero no sustituye una evaluación en circuitos, cámaras e iluminaciones no vistas.

La Fig. 1 muestra una predicción representativa del sistema. La máscara verde proporciona la referencia para calcular el centro de la trayectoria y la máscara roja activa la lógica de precaución ante obstáculos. El uso de una salida multicanal facilita que ambas decisiones procedan de una misma pasada de inferencia, lo que simplifica la sincronización entre percepción y control. Además, el postprocesado morfológico reduce pequeñas discontinuidades en la máscara del círculo antes de extraer el centro.

En la integración, el sistema queda preparado para operar con arranque coordinado desde el portátil. El script principal sincroniza los componentes necesarios en la Jetson, inicia el

emisor de vídeo, levanta el servidor de control remoto y abre el receptor local con la interfaz de visión. Esta organización permitió aislar responsabilidades: la Jetson conserva la interacción con hardware y el portátil concentra la inferencia y la depuración visual. No se registraron en el repositorio medidas experimentales cerradas de latencia o FPS real, por lo que esos valores se documentan como configuración objetivo y no como resultado medido.

V. DISCUSIÓN

El enfoque es adecuado para robótica educativa porque la segmentación semántica ofrece una señal de control interpretable: las máscaras pueden visualizarse y compararse con anotaciones. ONNX desacopla entrenamiento y despliegue, mientras que el reparto Jetson-portátil reduce la carga embebida. Las principales limitaciones son el tamaño del dataset y la dependencia del entorno: la red podría degradarse ante suelos, brillos, cámaras o posiciones de obstáculos no vistas. Queda pendiente medir latencia extremo a extremo, FPS real, estabilidad del seguimiento e intervenciones manuales por vuelta.

VI. CONCLUSIONES

Se ha desarrollado un sistema funcional de percepción y control para un vehículo robot con Jetson Nano. La solución integra un dataset propio, una U-Net ligera de dos canales, exportación ONNX y un runtime distribuido que transforma máscaras en comandos de *steering* y *throttle*. El Dice de validación de 0.9519 confirma la viabilidad del enfoque en el escenario planteado. Como trabajo futuro se propone ampliar datos, optimizar con TensorRT y medir latencia, FPS y robustez.

REFERENCIAS

- [1] Material docente de Visión Avanzada, “Segmentación con DeepLabv3+,” Tema 5, 2026.
- [2] L.-C. Chen, G. Papandreou, F. Schroff y H. Adam, “Rethinking Atrous Convolution for Semantic Image Segmentation,” arXiv:1706.05587, 2017.
- [3] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff y H. Adam, “Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation,” en ECCV, 2018.
- [4] O. Ronneberger, P. Fischer y T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” en MICCAI, 2015.
- [5] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” en NeurIPS, 2019.